Algorytmy blokowe ćwiczenie 3

Agata Wójtowicz

25 listopada 2025

1 Generowanie kluczy rundowych

1.1 Opis algorytmu

Proces generowania kluczy rundowych wykorzystuje algorytm szyfru Serpent. Do zaprezentowania przebiegu algorytmu, wykorzystany został 128-bitowy klucz główny:

$$\text{userKey} = 770A8A65DA156D24EE2A093277530142 \text{ (hex)}$$

Na początku klucz dzielony jest na 4 części 32-bitowe.

$$hex : \text{keyWords} = [770a8a65, \quad da156d24, \quad ee2a0932, \quad 77530142]$$

$$dec : k_1 = 1997179493 \qquad k_2 = 3658837284 \qquad k_3 = 3995732274 \qquad k_4 = 2001928514$$

Na podstawie wartości k_1, k_2, k_3, k_4 generowane jest 132 wartości preKeys zgodnie z podanym wzorem:

$$w_i = w_{i-4} \oplus w_{i-3} \oplus w_{i-2} \oplus w_{i-1} \oplus \varphi \oplus i11$$

$$\varphi = \frac{\sqrt{5}+1}{2}$$

Cztery pierwsze wartości w_i odpowiadają słowom k_1, k_2, k_3, k_4. Aby wyznaczyć kolejne wartości, wykorzystana została zamiana ciągu 32-bitowego na wektor w ciele $GF(2)$, w którym operacja xor to dodawanie wektorów. Po wyznaczaniu wyniku równania, wartość wektorowa zamieniana jest z powrotem na zapis w postaci liczby całkowitej i wykonywana jest operacja przesunięcia bitowego o 11 bitów w lewo. Obliczenia dla w_4:

$$\begin{aligned}
w_4 &= w_0 \oplus w_1 \oplus w_2 \oplus w_3 \oplus \varphi \oplus 4 \\
w_4 &= [1,0,1,0,0,1,1,0,0,1,0,1,0,0,0,1,0,1,0,1,0,0,0,0,1,1,1,0,1,1,1,0] \\
w_3 &= [0,0,1,0,0,1,0,0,1,0,1,1,0,1,1,0,1,0,1,0,1,0,0,0,0,1,0,1,1,0,1,1] \\
w_2 &= [0,1,0,4,1,1,0,0,1,0,0,1,0,0,0,0,0,1,0,1,0,1,0,0,0,1,1,1,0,1,1,1] \\
w_1 &= [0,1,0,0,0,0,1,0,1,0,0,0,0,0,0,0,1,1,0,0,1,0,1,0,1,1,1,0,1,1,1,0] \\
\varphi &= [1,0,0,1,1,1,0,1,1,0,0,1,1,1,1,0,1,1,1,0,1,1,0,0,0,1,1,1,1,0,0,1] \\
i &= [0,0,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0] \\
\text{wynik} &= [0,0,1,1,0,0,0,1,0,1,1,0,1,0,0,1,1,0,0,0,1,0,1,0,0,1,0,1,0,1,0,1] \\
w_4 &= [(0,1,0,0,1,0,1,0,1,0,1,0,0,1,1,0,0,0,1,0,1,1,0,1,0,0,1,1,0,0,0,1)] = 2360632658 = 8\text{cb}46552 \text{ (hex)}
\end{aligned}$$

W kolejnym kroku generowane są podklucze (subKeys). Cztery wyniki preKeys (w_0, w_1, w_2, w_3) przetwarzane są jednocześnie za pomocą tego samego Sboxa i zwracane są 4 podklucze k_0, k_1, k_2, k_3. Wykorzystywane jest 8 różnych Sboxów w ustalonej kolejności.

$$\begin{aligned}
\{k_0, k_1, k_2, k_3\} &:= S_3(w_0, w_1, w_2, w_3) \\
\{k_4, k_5, k_6, k_7\} &:= S_2(w_4, w_5, w_6, w_7) \\
\{k_8, k_9, k_{10}, k_{11}\} &:= S_1(w_8, w_9, w_{10}, w_{11}) \\
&\dots \\
\{k_{128}, k_{129}, k_{130}, k_{131}\} &:= S_0(w_{128}, w_{129}, w_{130}, w_{131})
\end{aligned}$$

Kolejność wykorzystywanych Sboxów: $[S_3, S_2, S_1, S_0, S_7, S_5, S_6, S_4]$. Wykorzystane Sboxy zostały wygenerowane na podstawie funkcji wykorzystanej podczas ćwiczenia nr 1. Wszystkie Sboxy generowane są za pomocą tych samych przekształceń, ale różnią się elementami początkowymi.

$$
\begin{aligned}
S_0 &= [12, 13, 0, 4, 10, 3, 8, 2, 15, 1, 7, 6, 14, 5, 11, 9] \\
S_1 &= [15, 14, 3, 7, 9, 0, 11, 1, 12, 2, 4, 5, 13, 6, 8, 10] \\
S_2 &= [3, 2, 15, 11, 5, 12, 7, 13, 0, 14, 8, 9, 1, 10, 4, 6] \\
S_3 &= [4, 5, 8, 12, 2, 11, 0, 10, 7, 9, 15, 14, 6, 13, 3, 1] \\
S_4 &= [11, 10, 7, 3, 13, 4, 15, 5, 8, 6, 0, 1, 9, 2, 12, 14] \\
S_5 &= [8, 9, 4, 0, 14, 7, 12, 6, 11, 5, 3, 2, 10, 1, 15, 13] \\
S_6 &= [7, 6, 11, 15, 1, 8, 3, 9, 4, 10, 12, 13, 5, 14, 0, 2] \\
S_7 &= [9, 8, 5, 1, 15, 6, 13, 7, 10, 4, 2, 3, 11, 0, 14, 12]
\end{aligned}
$$

Przykład wykorzystania Sboxów dla: (k_4, k_5, k_6, k_7).

Na wejściu: $(w_4, w_5, w_6, w_7) = (\text{0x8cb46552}, \text{0x7bcdd28f}, \text{0xbe309781}, \text{0x6ac50501})$

Wyznaczany jest klucz rundy 2, dlatego wykorzystywany jest sbox S_2 (zgodnie z kolejnością).
Zaimplementowane Sboxy są 4-bitowe na wejściu 32-bitowym, a więc do wyznaczenia jednej wartości k_i wykorzystywane jest 8 tych samych Sboxów równolegle.

Permutacja z wykorzystaniem S_2

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
3	2	15	11	5	12	7	13	0	14	8	9	1	10	4	6

$$
\begin{aligned}
S_2(\text{8cb46552}) &= \text{01957ccf} \\
S_2(\text{7bcdd28f}) &= \text{d91aaf06} \\
S_2(\text{be309781}) &= \text{94b3ed02} \\
S_2(\text{6ac50501}) &= \text{781c3c32}
\end{aligned}
$$

$$(k_4, k_5, k_6, k_7) = (\text{01957ccf}, \text{d91aaf06}, \text{94b3ed02}, \text{781c3c32})$$

Ostatnim krokiem jest wygenerowanie 33 128-bitowych kluczy rundowych. Każdy z kluczy rundowych K_i powstaje przez konkatenację wartości $k_{4i}, k_{4i+1}, k_{4i+2}, k_{4i+3}$.

2 Pseudokod

Algorithm 1 Generowanie kluczy rundowych

Require: K
Ensure: $roundKeys = [\ ]$

```
keyWords ← newArray[4]
for i ← 0 to 3 do
    shift ← 96 − (i ∗ 32)
    keyWords[i] ← (K ≫ shift) & 0xFFFFFFFF
end for
phi ← wektor((sqrt(5) + 1)/2 ∗ 2^(2∗32) & 0xFFFFFFFF)
for i ← 4 to 132 do
    v_44 ← wektor(keyWords[i − 4])
    v_43 ← wektor(keyWords[i − 3])
    v_42 ← wektor(keyWords[i − 2])
    v_11 ← wektor(keyWords[i − 1])
    v_i ← wektor(keyWords[i])
    wynik ← v_44 + v_13 + v_12 + v_11 + phi + v_i
    w_i ← int(wynik11 & 0xFFFFFFFF)
    keyWord[i] ← w_i
end for
subKeys ← [ ]
sbox_order ← [3, 2, 1, 0, 7, 5, 6, 4]
for i ← 0 to 33 do
    if i < len(sbox_order) then
        sbox_index ← sbox_order[i]
    else
        sbox_index ← sbox_order[i mod 8]
    end if
    start ← i ∗ 4
    sbox_input = keyWord[start : start + 4]
    for w_i in sbox_input do
        subKey ← sbox_opperation(sbox, w_i)
        subKeys[i] ← subKey
    end for
end for
roundKeys ← [ ]
for i in range(0, len(subKeys), 4) do
    k0, k1, k2, k3 ← subKeys[i], subKeys[i + 1], subKeys[i + 2], subKeys[i + 3]
    roundKey ← (k0 ≪ 96)|(k1 ≪ 64)|(k2 ≪ 32)|k3
    roundKeys[i] ← roundKey
end for
```

W tym algorytmie wykonanie operacji przy pomocy Sboxa zostało opisane jako `sbox_opperation()`. Poniżej algorytm wykorzystania 4-bitowego Sboxa na 32-bitowym ciągu w_i.

Algorithm 2 Wykonanie operacji sbox

Require: $sbox, w_i$
Ensure: $result$

```
result ← 0
for j in range(0, 32, 4) do
    oneSbox ← (w_i ≫ j) & 0xF
    sbox_output ← sbox[oneSbox]
    result ← result|(sbox_output ≪ j)
end for
```

3 Wygenerowane klucze rundowe

Klucz główny: 770A8A65DA156D24EE2A093277530142

$$
\begin{aligned}
K_0 &= aa4f7f0bdf5b0d82338f49c8aabc4528 \\
K_1 &= 01957ccfd91aaf0694b3ed02781c3c32 \\
K_2 &= 6481b0869383210a7cddf82b6df9544e \\
K_3 &= 5d91f2d208bb39a9ac7bea893891c27e \\
K_4 &= 338841c3c33dca2b1ea4cefd976a3037 \\
K_5 &= 83bb7d03aeff732c89df50ea235ca326 \\
K_6 &= 5ab0d98a203ec61106dbb4a7413559ab \\
K_7 &= 4db42ff4beb93089b327f4b5ba1c27ad \\
K_8 &= 955c53cf8ab53758696b45b4a83aea44 \\
K_9 &= 9e16f1e8c0c8fca137b554f5015643c9 \\
K_{10} &= d8a09148e64d8de14583487ced83970b \\
K_{11} &= 2d97d254c621900674ffc7159875075b \\
K_{12} &= a0e846ab68c1d5db9678f93abc59d6ac \\
K_{13} &= 37731ee06d7e378ce9cdfa602e852f70 \\
K_{14} &= 2ce9b84ca88a39932d1c24b3ef048e29
\end{aligned}
$$

4 Analiza odzyskania pełnego klucza ostatniej rundy

Krok 1:

Podział klucza rundowego na słowa (k_3, k_2, k_1, k_0).

$$K_{32} = 9ff848675de990f4770f260294e87c5b$$

$$k_{128} = 9ff84867 \quad k_{129} = 5de990f4 \quad k_{130} = 770f2602 \quad k_{131} = 94e87c5b$$

Krok 2:

Odzyskanie ostatnich 4 preKeys:

- Wyznaczenie indeksu Sboxa, który został wykorzystany w danej rundzie

  $sbox_index = 32 \pmod 8 = 0$
  $sbox_order = [S_3, S_2, S_1, S_0, S_7, S_5, S_6, S_4]$

  W rundzie 32 wykorzystany został Sbox pod indeksem 0, czyli S_3.

- Wykonanie operacji odwrotnej do `sbox_opperation`, aby odzyskać wartości $(w_{131}, w_{130}, w_{129}, w_{128})$.

Permutacja odwrotna do Sboxa S_3

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
6	15	4	14	0	1	12	8	2	9	7	5	3	13	11	10

$$
\begin{aligned}
w_{131} &= S_3^{-1}(k_{131}) = S_3^{-1}(94e87c5b) = 90b28315 \\
w_{130} &= S_3^{-1}(k_{130}) = S_3^{-1}(770f2602) = 886a4c64 \\
w_{129} &= S_3^{-1}(k_{129}) = S_3^{-1}(5de990f4) = 1db996a0 \\
w_{128} &= S_3^{-1}(k_{128}) = S_3^{-1}(9ff84867) = 9aa202c8
\end{aligned}
$$

Krok 3:

Odzyskanie poprzednich wartości preKeys na podstawie $(w_{131}, w_{130}, w_{129}, w_{128})$. Znając wartości $w_i, w_{i-3}, w_{i-2}, w_{i-1}$ można obliczyć wartość w_{i-4}. Kolejne wartości preKeys wyznaczamy iteracyjnie poprzez:

- Odwrócenie operacji przesunięcia bitowego: $w_xor_rotated = (w_i \gg 11)|(w_i \ll 21)$
- Odwrócenie działania wielomianu: $w_{i-4} = w_xor_rotated \oplus w_{i-3} \oplus w_{i-2} \oplus w_{i-1} \oplus phi \oplus (i-4)$

Pierwsza iteracja ($i = 131$):

$$\begin{aligned}
w_{131} &= 90628315_{\text{hex}} = 10010000101100101000001100010101_{\text{bin}} \\
w_xor_rotated &= 62b21650_{\text{hex}} = 11000101011001000010110010100 00_{\text{bin}} \\
w_{127} &= w_xor_rotated \oplus w_{130} \oplus w_{129} \oplus w_{128} \oplus phi \oplus 127 \\
w_{127} &= 62b21650 \oplus 886a4c64 \oplus 1db996a0 \oplus 9aa202c8 \oplus 9e3779b9 \oplus 7f \\
w_{127} &= f3fdbf66
\end{aligned}$$

Dla kolejnych iteracji:

$$\begin{aligned}
w_{126} &= 6649577c \\
w_{125} &= 452b2cd8 \\
w_{124} &= 17b2e1bb \\
&\dots
\end{aligned}$$

Krok 4:

Wykonanie operacji Sbox na odzyskanych preKeys $(w_0, w_1, \dots, w_{131})$ i połączenie odzyskanych wartości w klucze rundowe.

Dla rundy nr 31:

$sbox_index = 31 \pmod 8 = 7$
$sbox_order = [S_3, S_2, S_1, S_0, S_7, S_5, S_6, S_4]$

W rundzie 31 wykorzystany został Sbox pod indeksem 7, czyli S_4.

Permutacja Sboxa S_4

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
11	10	7	3	13	4	15	5	8	6	0	1	9	2	12	14

$$\begin{aligned}
k_{127} &= S_4(w_{127}) = S_4(f3f4b766) = e3ed15ff \\
k_{126} &= S_4(w_{126}) = S_4(6649577c) = ffd64559 \\
k_{125} &= S_4(w_{125}) = S_4(452b2cd8) = d4717928 \\
k_{124} &= S_4(w_{124}) = S_4(17b2e1bb) = a517ca11
\end{aligned}$$

Klucz rundowy: a517ca11d4717928ffd64559e3ed15ff
Odzyskanie kolejnych kluczy rundowych przebiega analogicznie do przypadku przeanalizowanego dla klucza rundy 31.

Wniosek:

Na podstawie wartości $w_{131}, w_{130}, w_{129}, w_{128}$ można wyznaczyć wszystkie poprzednie. Następnie dla każdych czterech znalezionych wartości wystarczy przeprowadzić permutację przy pomocy odpowiedniego Sboxa. Znając cały klucz ostatniej rundy, można odzyskać wszystkie poprzednie klucze rundowe.

5 Analiza odzyskania połowy klucza ostatniej rundy

5.1 Założenie 1

W pierwszym założeniu, analizowana jest sytuacja, w której znana jest połowa bitów od 1 do 64.

$$K_{32} : [9, f, f, 8, 4, 8, 6, 7, 5, d, e, 9, 9, 0, f, 4, X, X, X, X, X, X, X, X, X, X, X, X, X, X]$$

Na podstawie znanych bitów:

$$k_{128} = 9ff84867$$
$$k_{129} = 5de990f4$$

Można wyznaczyć w_{128} oraz w_{129}

- Wyznaczenie indeksu Sboxa, który został wykorzystany w danej rundzie

  $sbox_index = 32 \pmod 8 = 0$
  $sbox_order = [S_3, S_2, S_1, S_0, S_7, S_5, S_6, S_4]$

  W rundzie 32 wykorzystany został Sbox pod indeksem 0, czyli S_3.

- Wykonanie operacji odwrotnej do `sbox_opperation`, aby odzyskać wartości $(w_{131}, w_{130}, w_{129}, w_{128})$.

Permutacja odwrotna do Sboxa S_3

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
6	15	4	14	0	1	12	8	2	9	7	5	3	13	11	10

$$w_{128} = S_3^{-1}(k_{128}) = 9aa202c8$$
$$w_{129} = S_3^{-1}(k_{129}) = 1db996a0$$

Odzyskanie klucza poprzedniej rundy jest możliwe, tylko po przez odwrócenie działania wielomianu. W tym przypadku, dwa wyrazy nie są znane.

$$w_{127} = w_{131} \oplus w_{130} \oplus w_{128} \oplus \phi \oplus 127$$

$$w_{127} = (w_{127} >> 11 | w_{127} << 21)$$

Wyznaczanie wyrazów w_{131} i w_{130} nie jest możliwe, ponieważ połowa klucza nie jest znana, stąd w_{127} oraz każda kolejna wartość $preKeys$ jest niedostępna. Bez znajomości wartości $preKeys$, nie można odzyskać żadnego z bitów poprzednich kluczy rundowych.

5.2 Założenie 2

W pierwszym założeniu, analizowana jest sytuacja, w której znana jest połowa bitów od 64 do 128.

$$K_{32} : [X, X, X, X, X, X, X, X, X, X, X, X, X, X, X, X, 7, 7, 0, f, 2, 6, 0, 2, 9, 4, e, 8, 7, c, 5, b]$$

Sytuacja jest analogiczna do poprzedniego założenia. W tym przypadku można odzyskać wartości w_{130} oraz w_{131}, ale nie można odwrócić działania wielomianu, bez znajomości dwóch arguemntów. Niezależenie od tego, która część klucza jest znana, nie można odzyskać żadnego z poprzednich kluczy rundowych.

5.3 Założenie 3

W trzecim założeniu, analizowana jest sytuacja, w której znany jest co czwarty bit klucza.

$$K_{32} : [9, X, f, X, 4, X, 6, X, 5, X, e, X, 9, X, f, X, 7, X, 0, X, 2, X, 0, X, 9, X, e, X, 7, X, 5]$$

W tym przypadku znamy połowę każdego ze słów składającego się na klucz.

$$k_{128} = 9XfX4X6X$$
$$k_{129} = 5XeX9XfX$$
$$k_{127} = 7X0X2X0X$$
$$k_{126} = 9XeX7X5X$$

Na znanych bitach można wykonać odwrotną permutację do Sboxa S_3 tak jak w poprzednich założeniach.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
6	15	4	14	0	1	12	8	2	9	7	5	3	13	11	10

$$w_{128} = S_3^{-1}(k_{128}) = 9XaX0XcX$$
$$w_{129} = S_3^{-1}(k_{129}) = 1XbX9XaX$$
$$w_{130} = S_3^{-1}(k_{130}) = 8X6X4X6X$$
$$w_{131} = S_3^{-1}(k_{131}) = 9XbX8X1X$$

Znając połowę bitów każdego z wyrazów wielomianu, również nie można go odwrócić. Stąd możemy odzyskać połowę z czterech wartości $preKeys$, ale w tym założeniu, odzyskanie poprzednich kluczy runodwych również jest niemozliwe.

6 Wnioski

1. **Wykorzystanie Sboxów**
   Konstrukcja Sbox jest jednoznaczna i odwracalna. Jednoznaczność jest ważna dla procesu generowania kluczy, ponieważ te same dane wejściowe zawsze dadzą te same klucze rund. Odwracalność tej operacji jest istotna dla przeprowadzenia kryptoanalizy. Z podkluczy $k_{128} \dots k_{131}$ odzyskać dokładnie te same wartości $w_{128} \dots w_{131}$, które posłużyły do ich wygenerowania. Sbox-y utrudniają znalezienie stałej charakterystyki między kluczami rundowymi. Zmiany w kluczu głównym propagują się na wszystkie podklucze.

2. **Funkcja afiniczna - rekurencyjne wyznaczanie kolejnych elementów klucza.**
   Struktura z wykorzystaniem wielomianu decyduje o braku odporności algorytmu przy znajomości pełnego klucza ostatniej rundy oraz odporności przy odzyskaniu danych częściowych. Wyznaczanie wartości w_i, wymaga znajomości wszystkich czterech wartości wielomianu.